



U4 - Despliegues en AWS

2º DAW

2025/2026

Juanra Collado – jr.colladoesteve@edu.gva.es

RA's que se trabajan en esta unidad

- RA1 - Implanta arquitecturas web analizando y aplicando criterios de funcionalidad
 - Este RA también se trabaja en la U1 y U2.
 - RA2 - Implanta aplicaciones web en servidores web, evaluando y aplicando criterios de configuración para su funcionamiento seguro.
 - Este RA también se trabaja en la U1, U2 y U3.
 - RA3 - Implanta aplicaciones web en servidores de aplicaciones, evaluando y aplicando criterios de configuración para su funcionamiento seguro.
 - Este RA también se trabaja en la U3.
-

De local a cloud

- Hasta ahora, en la U1, aprendisteis a desplegar webs en máquinas locales (VirtualBox) y VPS tradicionales (PIM).
- En esta unidad veremos cómo llevar ese mismo proceso a la nube.
- En lugar de instalar y mantener el hardware, la nube ofrece los recursos como servicio:
 - Contratamos lo que necesitamos y nos permite ampliar según nuestras necesidades

¿Qué es AWS?

- **AWS (Amazon Web Services)** es el proveedor líder de servicios cloud a nivel mundial.
- Permite crear servidores, bases de datos, almacenamiento, redes y herramientas DevOps bajo demanda.
- Funciona con modelo **pago por uso**: solo se paga por el tiempo o recursos utilizados.



Ventajas principales

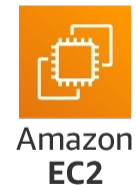
- Ventajas principales:
 - Escalabilidad inmediata (aumentar o reducir capacidad según carga).
 - Alta disponibilidad y redundancia entre zonas geográficas.
 - Seguridad y control de acceso centralizado (IAM, SG).
 - Automatización y despliegue continuo (más adelante en U5).

Regiones y zona de disponibilidad

- AWS divide su infraestructura global en **regiones** (por ejemplo, *eu-west-1* = Irlanda).
 - Cada región contiene varias **zonas de disponibilidad (AZ)**, que son centros de datos independientes.
 - Al lanzar recursos (como una instancia EC2), siempre debemos elegir la región y la AZ.
 - En AWS Academy normalmente se usa **us-east-1 (N. Virginia)**.
-

Introducción a EC2

- **Amazon EC2 (Elastic Compute Cloud)** ofrece máquinas virtuales en la nube.
- Una instancia EC2 es el equivalente cloud de la máquina virtual que usabas en VirtualBox.
- Permite elegir:
 - Tipo de sistema operativo (Linux, Windows).
 - Tamaño de hardware (CPU, RAM, disco).
 - Clave SSH para acceso seguro.
 - ...



Conexión y despliegue

- Las instancias EC2 no tienen interfaz gráfica: se gestionan por **SSH (Linux)** o **RDP (Windows)**.
- Tras conectarse, se pueden instalar los mismos servicios que ya conoces: Apache, Nginx, PHP, etc.
- El flujo básico será:
 - Crear instancia EC2.
 - Conectarse por SSH.
 - Instalar servidor web.
 - Probar acceso desde navegador.

Práctica 1: Conexión a AWS

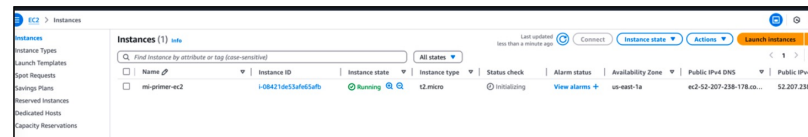
- Habremos recibido un correo de AWS para darnos de alta en AWS Academy.
 - Si no lo has hecho, contacta con el profesor.
 - Seguiremos el proceso exacto descrito en esta web, teniendo en cuenta que para este módulo solo tendremos el laboratorio y no dos cursos como explica allí:
 - https://juanracollado.github.io/cloud/u1_practiques/#que-es-un-aws-academy-learner-lab
 - Una vez hemos accedido al laboratorio y estamos en la consola de AWS, podemos continuar con la siguiente diapositiva.
-

Práctica 2: Creación de un EC2

- Para crear un EC2, seguiremos los pasos de la práctica 2 de la siguiente web (sigue leyendo):
 - https://juanracollado.github.io/cloud/u2_practiques/#practica-2-creacio-de-la-primera-instancia-ec2
- En el paso “Network Settings” es importante que seleccionemos también *Allow HTTP traffic from the Internet*

Práctica 2: Creación de un EC2

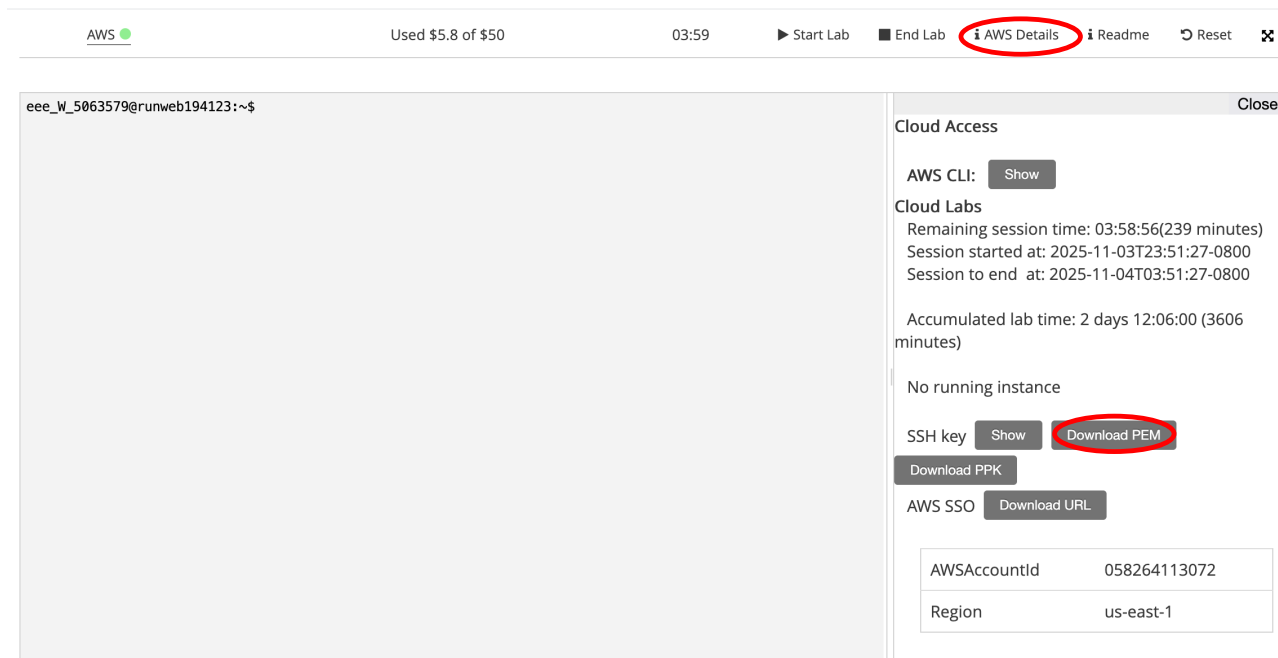
- Como resultado, tendremos nuestra primera instancia corriendo y funcionando en el cloud:



Práctica 3: Conexión por SSH a EC2

- Para conectarnos por SSH a nuestra instancia EC2 que acabamos de crear, necesitaremos descargar la clave mediante la cual accederemos:
 - EC2 no funciona con usuario y contraseña sino con par de claves.
 - En nuestro caso, al ser un entorno de pruebas, el par de claves se descarga desde la pantalla desde donde abrimos el laboratorio.
 - Para ello iremos a la pestaña *AWS Details* y pulsaremos el botón *Download PEM*
 - Si quisiéramos acceder desde Windows o desde Filezilla, necesitaríamos el PPK. Para SSH desde Linux solamente necesitamos la clave PEM.
-

Práctica 3: Conexión por SSH a EC2



The screenshot displays the AWS Cloud Labs interface. At the top, a status bar shows 'AWS' with a green dot, 'Used \$5.8 of \$50', '03:59', and buttons for 'Start Lab', 'End Lab', 'AWS Details' (circled in red), 'Readme', 'Reset', and a close icon. The main area is split into two panes. The left pane shows a terminal window with the prompt 'eee_w_5063579@runweb194123:~\$'. The right pane, titled 'Cloud Access' with a 'Close' button, contains several sections: 'AWS CLI' with a 'Show' button; 'Cloud Labs' with session timing information (Remaining session time: 03:58:56(239 minutes), Session started at: 2025-11-03T23:51:27-0800, Session to end at: 2025-11-04T03:51:27-0800, and Accumulated lab time: 2 days 12:06:00 (3606 minutes)); 'No running instance'; 'SSH key' with 'Show' and 'Download PEM' (circled in red) buttons; 'Download PPK'; 'AWS SSO' with a 'Download URL' button; and a table with AWS account details.

AWSAccountId	058264113072
Region	us-east-1

Práctica 3: Conexión por SSH a EC2

- Una vez descargado el PEM, necesitamos setearlo permisos adecuados:
 - *sudo chmod 400 nombredelPEM.pem*
- Y ya podemos conectarnos a nuestra instancia EC2 por SSH:
 - *ssh -i nombredelPEM.pem nombreUsuario@IP_PUBLICA_EC2*
 - El nombre de usuario para Ubuntu es *ubuntu* y para Linux Ami es *ec2-user*
 - La ip pública se puede obtener de los detalles de la instancia

Práctica 4: Primera Web

- Una vez conectados, ya tratamos con el servidor exactamente igual que lo hacíamos con nuestra máquina virtual.
- Instalamos Apache
- Modificamos la web por defecto
- Comprobamos acceso
 - Para ello, desde el navegador accedemos a `http://IP_PUBLICA_EC2` y debemos ver la web que hemos desplegado.

Práctica 5

- Haz lo mismo con una AMI (SO) Amazon Linux (es un Fedora) y un servidor nginx.

Seguridad en AWS

- La seguridad en EC2 se gestiona mediante grupos de seguridad (SG)
 - Un **Security Group** (SG) es un **firewall virtual** que controla el tráfico de red que entra o sale de una instancia EC2.
 - Cada SG contiene **reglas de entrada (inbound)** y **reglas de salida (outbound)**.
 - Por defecto:
 - **Inbound:** todo bloqueado.
 - **Outbound:** todo permitido.
-

Seguridad en AWS

- Solo abriremos los puertos estrictamente necesarios.
- Algunos importantes:

Servicio	Puerto	Protocolo
SSH	22	TCP
HTTP	80	TCP
HTTPS	443	TCP
MySQL	3306	TCP
MongoDB	27017	TCP

Seguridad en AWS

- En la U1, vimos que había que abrir puertos a nivel de SO (ufw).
- SG vs Firewall interno

Concepto	Security Group (AWS)	ufw / iptables (SO)
Nivel	Red virtual de AWS	Dentro del sistema operativo
Alcance	Antes de llegar al servidor	Después de llegar
Configuración	En consola AWS	En terminal Linux
Uso recomendado	Control externo	Protección adicional interna

Seguridad en AWS

- Además del acceso a la red, AWS también permite gestionar el acceso a la consola de gestión (quién puede hacer qué)
- A través de IAM (Identity and Access Management) se crean usuarios y roles.
- En el entorno de AWS Academy esta parte está limitada.

Práctica 6

- Vamos a modificar los SG de la instancia creada en las prácticas 1-4 para que solo se pueda acceder por SSH desde nuestra IP.
 - Desde la consola, accedemos a EC2 y seleccionamos la instancia que nos interesa.
 - En la pestaña *Security->Security Groups* veremos:
 - SSH (22)->0.0.0.0/0
 - HTTP (80) -> 0.0.0.0/0
-

Práctica 6

- 0.0.0.0/0 significa que lo tenemos permitido para todo el mundo. Es decir, cualquiera puede conectarse a estos puertos de nuestra EC2.
 - Estas reglas las hemos definido en el apartado “Network Settings” al crear la EC2 pero se pueden modificar a posteriori.
 - Podemos modificarlo y poner una IP específica, una subred o nuestra propia IP.
 - Modificamos la regla SSH para que solo permita conectarse por SSH a *My IP*.
-

Práctica 6

- Si probamos el acceso por SSH, veremos que seguimos pudiendo conectarnos pero desde otro PC (IP) no nos debe dejar.
- Por HTTP seguimos pudiendo acceder a la Web.
- Elimina la regla del puerto 80 y comprueba que la web ha dejado de estar accesible a través del navegador.

Conexión FTP en AWS

- Podemos conectar por FTP (Filezilla, por ejemplo) a nuestra instancia EC2 pero hay que tener algunas cosas en cuenta:
 - Se utiliza el fichero ppk para conectar, no el pem, independientemente del SO utilizado.
 - Nos conectamos utilizando usuario ubuntu/ec2-user, por tanto, los permisos en algunas carpetas están limitados.
 - Una solución es mover los ficheros a carpetas donde tengamos permisos y luego moverlos mediante sudo, si lo vemos necesario.

Práctica 7

- Para conectarnos por FTP a EC2 seguiremos la siguiente práctica:
 - https://juanracollado.github.io/cloud/u2_practiques/#practica-6-connexio-per-ftp-utilitzant-el-filezilla
- Una vez conectados, podemos mover, por ejemplo, un fichero html a nuestra raíz del servidor y comprobar que éste funciona correctamente (recuerda que habría que tener permitido el puerto 80 en el SG de la instancia)

Aislamiento de recursos

- Una de las ventajas de AWS es que permite aislar recursos en la nube mediante GS y comunicación privada entre instancias (para eso existen las IP's privadas de las EC2)
 - En despliegues profesionales, la base de datos nunca está accesible desde Internet.
 - Se coloca en una **subred privada** o con reglas de acceso muy restrictivas. Solo tiene acceso el administrador de la BBDD y el “cliente” de ésta (el servidor web)
 - Ventajas:
 - Mayor seguridad.
 - Menor exposición a ataques.
 - Escalabilidad independiente (puedes aumentar recursos en la capa web o de BD).
-

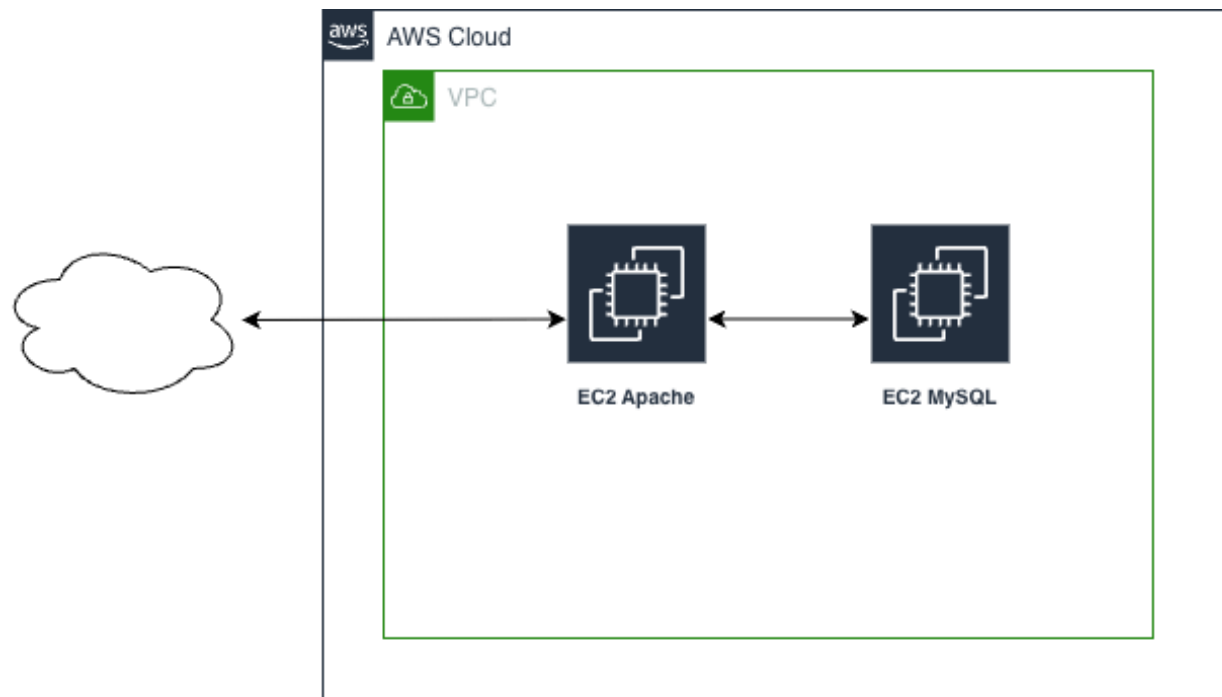
Práctica 8

- Crea una infraestructura con un servidor Web y una BBDD en dos EC2 diferentes.
 - El servidor web tendrá los puertos 22 (SSH) y 80 (HTTP) abiertos a todos
 - El servidor de BBDD tendrá solamente el puerto 22 y el puerto 3306 abiertos al servidor web (solo se podrá conectar desde el servidor web)
 - Para comprobar la conectividad puedes:
 - Conectarte por cliente al servidor MySQL
 - Crear un PHP que se conecte por código

Práctica 8

- En un punto determinado de la práctica, tendremos que conectarnos por SSH de una instancia a otra ya que la instancia de BD no permite conectarnos desde fuera de la red, solo desde la otra instancia
- A esta técnica se le llamaba reenvío de agente SSH.
- Podemos ver como se realiza en la documentación oficial o en la web de la optativa de la nube:
 - <https://repost.aws/es/knowledge-center/ec2-linux-private-subnet-bastion-host>
 - https://juanracollado.github.io/cloud/u3_practiques/#41-ssh-intern-ec2-ec2

Práctica 8



Práctica 8

- La solución a la práctica anterior está realizada en vídeo y lo tenéis disponible en Aules.
- Al finalizar la práctica no resetees el entorno ya que lo vas a necesitar para la práctica 10.

Amazon RDS

- **Amazon RDS (Relational Database Service)** es un servicio **gestionado** que permite usar motores de bases de datos como MySQL, PostgreSQL, MariaDB, SQL Server o Oracle sin administrar el sistema operativo ni el software de BD.
 - AWS se encarga de:
 - Instalación y parches.
 - Backups automáticos.
 - Replicación y alta disponibilidad.
 - Monitorización y escalado de recursos.
-

Amazon RDS

- RDS sacrifica control para ganar disponibilidad, fiabilidad y escalado automático.



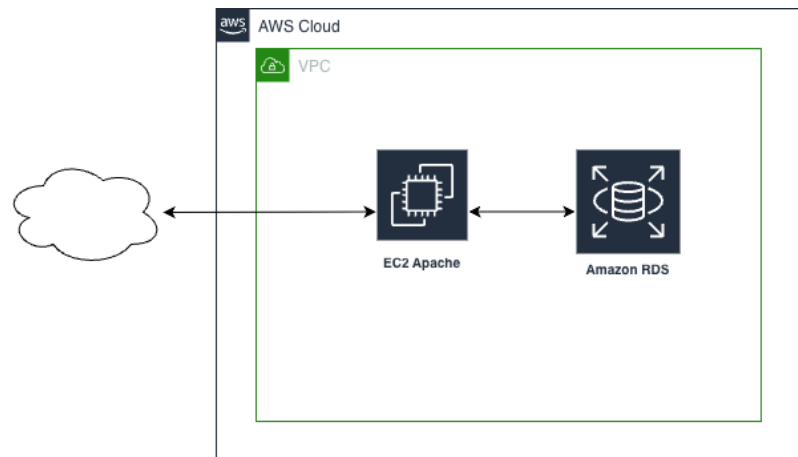
Amazon RDS

Práctica 9

- Crear una base de datos RDS es más sencillo ya que nos ahorramos el paso de crear la instancia EC2 y instalar MySQL.
 - Solamente hemos de crear la RDS y listo. Ya tendremos una IP a la que poder acceder como si hubiésemos instalado el MySQL en una instancia.
 - https://juanracollado.github.io/cloud/u4_practiques/#practica-8-creacio-i-configuracio-duna-instancia-rds
 - https://juanracollado.github.io/cloud/u4_practiques/#practica-9-connexio-a-la-base-de-dades-rds-per-consola-i-per-codi
 - https://juanracollado.github.io/cloud/u4_practiques/#practica-10-integracio-ec2-rds
-

Práctica 10

- Modifica la infraestructura creada en la práctica 8 para que la BD sea una RDS y no una EC2.



Despliegues en Windows IIS

- **IIS** es el servidor web oficial de Microsoft incluido en Windows Server.
 - Permite alojar sitios web, aplicaciones ASP.NET, PHP y servicios HTTP/HTTPS.
 - Cumple el mismo papel que Apache o Nginx, pero con interfaz gráfica y gestión integrada en Windows.
-



Despliegues en Windows IIS

- Trabajar con AWS es una oportunidad fantástica para poder conocer de primera mano el servidor de aplicaciones Web de Windows ya que éste necesita de más recursos que en los PC's de clase se hace complicado conseguir.
- Es por ello, que aprovecharemos esta unidad para estudiar de forma muy básica como funcionan los despliegues de aplicaciones web en el servidor IIS de Windows

Flujo para desplegar IIS en AWS

- El flujo de trabajo no es diferente al de despliegue en otro servidor web:
 - Crear instancia Windows en AWS.
 - Instalar y habilitar IIS.
 - Configurar carpeta raíz y probar página HTML o PHP.
 - Abrir puerto 80 o el que se defina.
 - Acceder desde navegador externo.

¿Cuándo utilizar IIS?

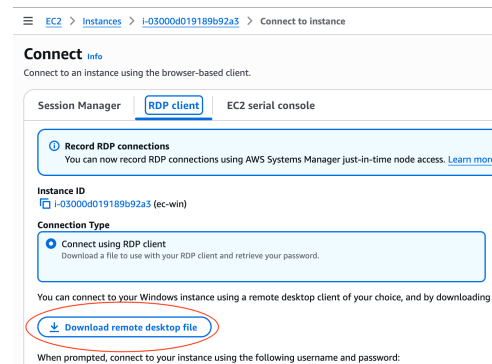
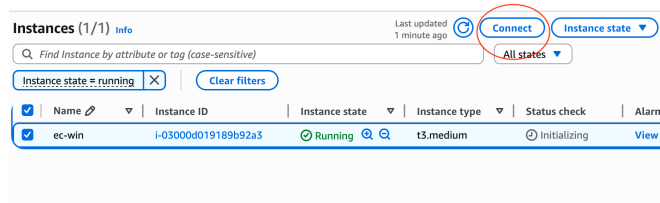
- Aplicaciones desarrolladas en **ASP.NET** o **.NET Core**.
 - Son tecnologías de Microsoft y por tanto, si o si, han de correr en un IIS.
 - Entornos mixtos en empresas que ya trabajan con Windows Server.
 - Adaptabilidad al entorno. Si todo está ya en Windows, tiene más sentido que si vamos a poner un servidor Web, sea sobre Windows.
 - Casos donde se requiere integración con **Active Directory** o servicios de Windows.
-

Práctica 11

- En esta práctica realizaremos un despliegue de una página web básica sobre IIS en AWS.
 - Lanzaremos una instancia basada en la AMI Windows Server 2025 base.
 - Seleccionaremos la t3.small o incluso la medium (cuidado que esta nos va a cobrar y por tanto tenemos que asegurarnos de apagarla nada más acabemos)
 - Al ser un entorno gráfico, necesitamos más RAM, por eso elegimos más recursos.
 - Abriremos SSH y HTTP
 - Lanzamos la instancia

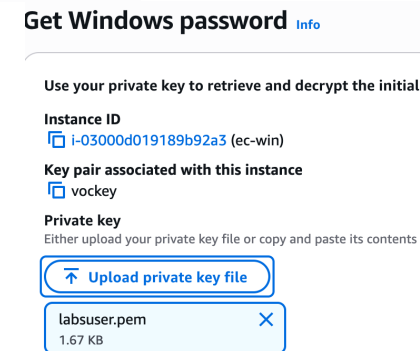
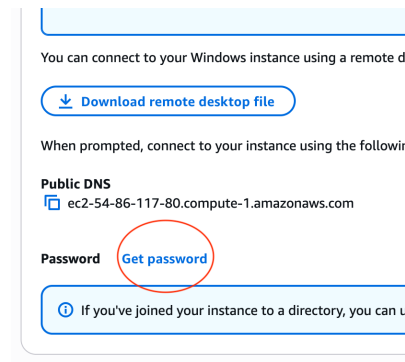
Práctica 11

- Para acceder a esta instancia, lo haremos por RDP (Remote Desktop Protocol).
 - Para ello, hacemos click en *connect* y después iremos a la pestaña RDP Client, donde seleccionaremos *Download remote desktop file*



Práctica 11

- Una vez el fichero descargado, seleccionaremos la opción *Get password*.
- Y en la siguiente pantalla subiremos nuestro .pem desde *Upload private key file*



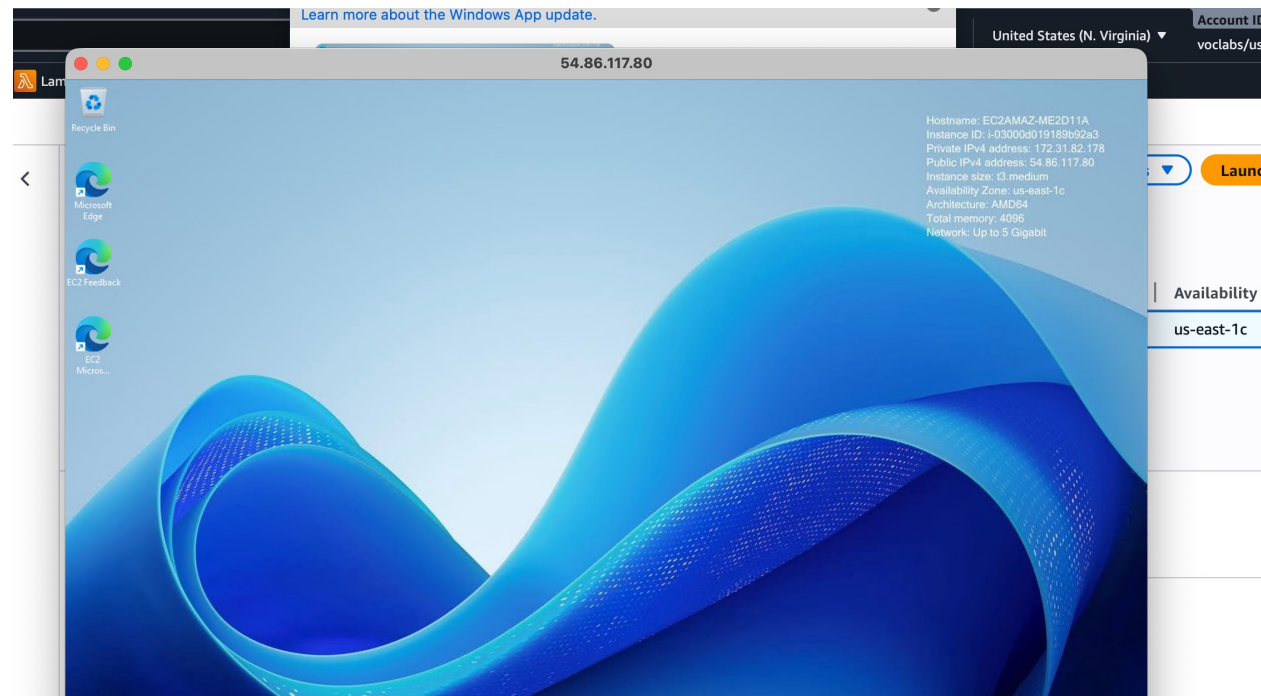
Práctica 11

- Tras subirlo, seleccionamos *Decrypt Password* y copiamos el password que nos da porque lo utilizaremos para conectarnos por RDP.
- Una vez hemos realizado todos estos pasos, podemos conectarnos utilizando una aplicación de escritorio remoto (por ejemplo, Microsoft Remote Desktop).
 - Si abrimos el fichero .rdp que hemos descargado, abrirá automáticamente una conexión al ec2 de Windows.

Práctica 11

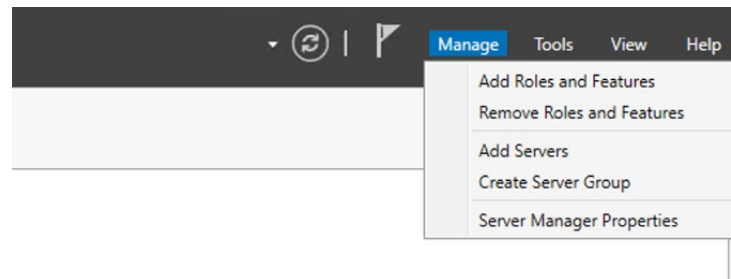
- Independientemente de la app utilizada, necesitaremos indicar para conectarnos:
 - IP pública de la instancia
 - Usuario (en Windows server es *Administrator*)
 - Password (lo hemos obtenido en el paso anterior)
- Una vez configurado, se nos conectará y mostrará el escritorio del Windows como si se una máquina virtual instalada en local se tratara.

Práctica 11



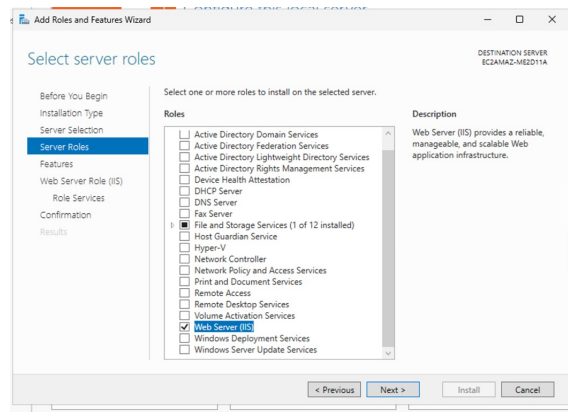
Práctica 11

- Una vez el WS funcionando, es momento de configurar el IIS.
- Al tratarse de una versión Server de Windows, solo hemos de habilitar el servicio desde *Server Manager->Manage->Add roles and features*



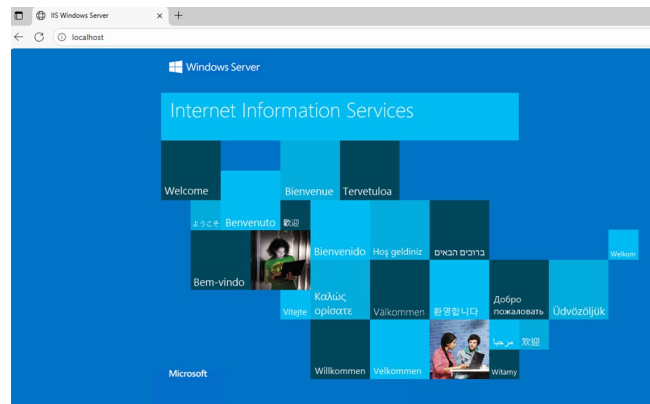
Práctica 11

- Se abrirá un asistente en el que indicaremos que llegaremos con todo por defecto hasta el apartado *Server roles* y marcaremos el servidor IIS y acabaremos la instalación.



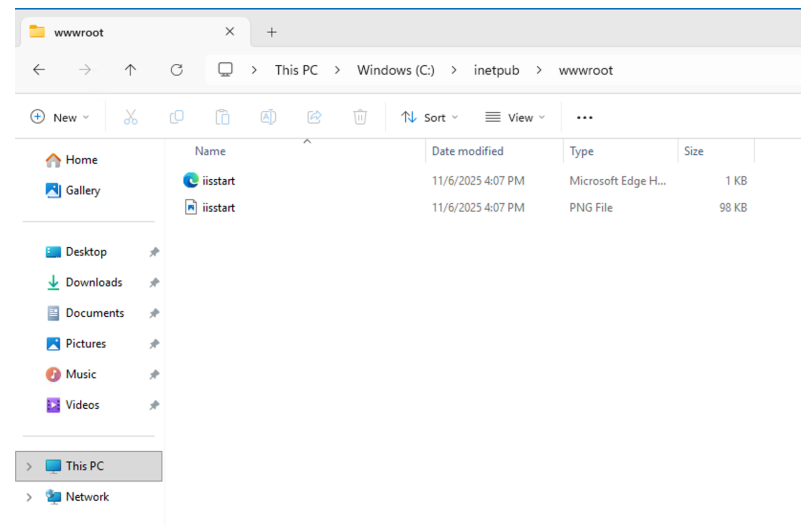
Práctica 11

- Una vez el ha finalizado la instalación, ya tendremos el servidor web funcionando en el puerto 80 y podremos acceder desde el Windows por localhost y desde cualquier máquina utilizando la IP pública de la instancia.



Práctica 11

- El DocumentRoot de IIS se encuentra en *C:\inetpub\wwwroot*



PEC 4

- En este punto, se recomienda realizar la PEC4.

Docker en AWS

- Otra forma muy utilizada para desplegar aplicaciones en la nube, es mediante el uso de contenedores.
- Su uso no difiere mucho de lo que aprendimos en local, mismos comandos, mismos conceptos.
 - En vez de lanzarse en local, se lanza en la instancia de EC2

Ventajas de Docker en la nube

- Despliegues homogéneos y portables.
 - Facilita la integración continua y el versionado.
 - Se estudia en la siguiente unidad
 - Reducción de errores por diferencias de entorno.
 - Base para usar servicios gestionados como **ECR (ECS Repository)** o **ECS Clusters** más adelante.
-

Flujo de trabajo

- EC2 con Docker instalado
- Creamos contenedor con run (por ejemplo)
 - Nos aseguramos de exponer y mapear el puerto correctamente
 - Nos aseguramos que el puerto que estamos mapeando esté accesible en el EC2.
 - Ojo! Hay que abrir en EC2 el puerto que mapeamos, no el que exponemos. Es decir, si hacemos un 9090:80, el que hay que exponer es el 9090, ya que es la puerta que utilizaremos nosotros para llegar al contenedor dentro del EC2.

Práctica 12

- Crea un EC2 con una AMI Ubuntu y deja abiertos los puertos 22(SSH) y el puerto 8080 (lo utilizaremos para mapearlo al 80 más adelante).
 - Siguiendo el manual oficial, instalaremos Docker Engine (yo he optado por el método de apt):
 - <https://docs.docker.com/engine/install/ubuntu/>
 - <https://docs.docker.com/engine/install/ubuntu/#install-using-the-repository>
 - Para comprobar que todo ha ido bien:
 - *docker versión*
 - *docker run hello-world*
-

Práctica 12

- Antes de continuar, ejecutaremos los pasos post-instalación:
 - <https://docs.docker.com/engine/install/linux-postinstall/>
 - A continuación, probaremos a ejecutar un contenedor para ver que funciona exactamente igual que en local. Para ello, replicaremos el ejemplo básico (octopus) de la unidad 2:
 - Clonamos el siguiente repositorio <https://github.com/JuanraCollado/octopus-underwater-app>
 - Creamos la imagen a partir del Dockerfile
 - Creamos el contenedor a partir de la imagen creada.
 - Recuerda que hay que mapear el puerto 80 del servidor nginx incluido en la imagen al puerto 8080 que es el que hemos dejado abierto al crear el EC2
-

Práctica 13

- Ahora crea una EC2 y despliega la misma web en nginx.
- Prueba a cambiar el puerto de nginx
 - Recuerda que hay que modificar los grupos de seguridad

Práctica 14

- Realiza el ejercicio 2 del PEC2 sobre AWS.

Docker Compose en AWS

- Herramienta que permite definir y ejecutar aplicaciones Docker multicontenedor mediante un solo archivo docker-compose.yml.
- Simplifica el trabajo con redes, dependencias y variables.
- En AWS lo utilizaremos igual que en local, ejecutándose dentro de la EC2.

Docker Compose en AWS

- Antes de empezar, nos aseguraremos que docker compose está correctamente instalado ejecutando:
 - *docker compose versión*
- En las últimas versiones se instala junto con el Docker.
 - Si no estuviese instalado, podemos hacerlo siguiendo el manual oficial
 - <https://docs.docker.com/compose/install/#plugin-linux-only>

Docker Compose en AWS

- Trabajar con compose ya sabemos y por tanto nos vamos a centrar en la migración de un entorno de docker-compose en local a un entorno en la nube (o cualquier VPS).
 - Esto simula un entorno profesional real, donde los desarrolladores trabajan en local con un entorno y quieren subirlo a “Producción”.
 - Gracias a Docker, este proceso es mucho más simple y seguro que antes.
 - Para ello, partiremos de un entorno en local que nos funciona perfectamente, como el que hicimos en el ejercicio 2 de la PEC3
 - Servidor Apache + PHP
 - MySQL
 - PHPMyAdmin
-

Docker Compose en AWS

- Hay que tener en cuenta que la migración a entornos “reales” se realiza una vez el desarrollo esté finalizado, o al menos una fase de éste.
- Es decir, no trabajaremos directamente sobre la nube como sí que lo hacíamos en local, sino que copiaremos los ficheros que tenemos en local a la nube.

Docker Compose en AWS

- ¿Esto significa que no tendremos bind mounts? NO.
- Los bind mounts son importantes ya que el entorno lo montamos una vez y si no lo cambiamos en local, podemos mantenerlo durante todo el tiempo e ir “subiendo” solamente los ficheros que van siendo modificados a la carpeta que tenemos sincronizada con el contenedor.
- Los conceptos son exactamente los mismos, solo hemos de cambiar la perspectiva:
 - Hasta ahora la sincronización era PC-Local-> Docker
 - Ahora será PC-EC2->Docker

Migración Local - AWS Docker

- Como hemos dicho, partiremos de la solución del ejercicio 2 de la PEC3.
 - Podéis coger vuestra solución
 - Coger la de aules
 - Encontraréis un Dockerfile para crear una imagen con las librerías para acceder a la BD
 - Y un docker-compose con esta imagen y las demás necesarias.
 - No es necesario hacer el build de la imagen por separado ya que éste se hace internamente en el docker-compose gracias al build interno.
 - Una vez nos hemos asegurado que el entorno funciona en local, procederemos a su migración a AWS
-

Migración Local - AWS Docker

- Dockerfile
 - Lo usaremos para el servicio que lanza el Apache con PHP ya instalado e instalaremos nosotros mediante el RUN la librería para conectarnos por PDO a la BD.

```
FROM php:8.3-apache

# Instala y habilita la extensión PDO MySQL
RUN docker-php-ext-install pdo_mysql

# EXPOSE 80 no es imprescindible, pero es informativo
EXPOSE 80
```


Migración Local - AWS Docker

- docker-compose.yml
 - Aparecen tres servicios
 - php_app
 - Se crea partiendo del docker file
 - Mapea el puerto 80 del contenedor al 8080
 - Por tanto, habrá que abrir en el EC2 el puerto 8080 para que por ahí puedan acceder al Apache.
 - Crea un bind volume de la carpeta web a la carpeta root de apache

```
app:
  build: .
  container_name: php_app
  environment:
    MYSQL_HOST: db
    MYSQL_USER: root
    MYSQL_PASSWORD: secret
    MYSQL_DATABASE: usuarios
  ports:
    - "8080:80"
  volumes:
    - ./web:/var/www/html
  depends_on:
    - db
  restart: always
```

Migración Local - AWS Docker

- mysql
 - Asocia el volumen administrado (que se crea al final del fichero) a la carpeta de datos de MySQL
- phpmyadmin
 - Mapea el puerto 80 del contenedor al 8081
 - Por tanto, habrá que abrir en el EC2 el puerto 8081 para que por ahí puedan acceder al PHPMYAdmin.

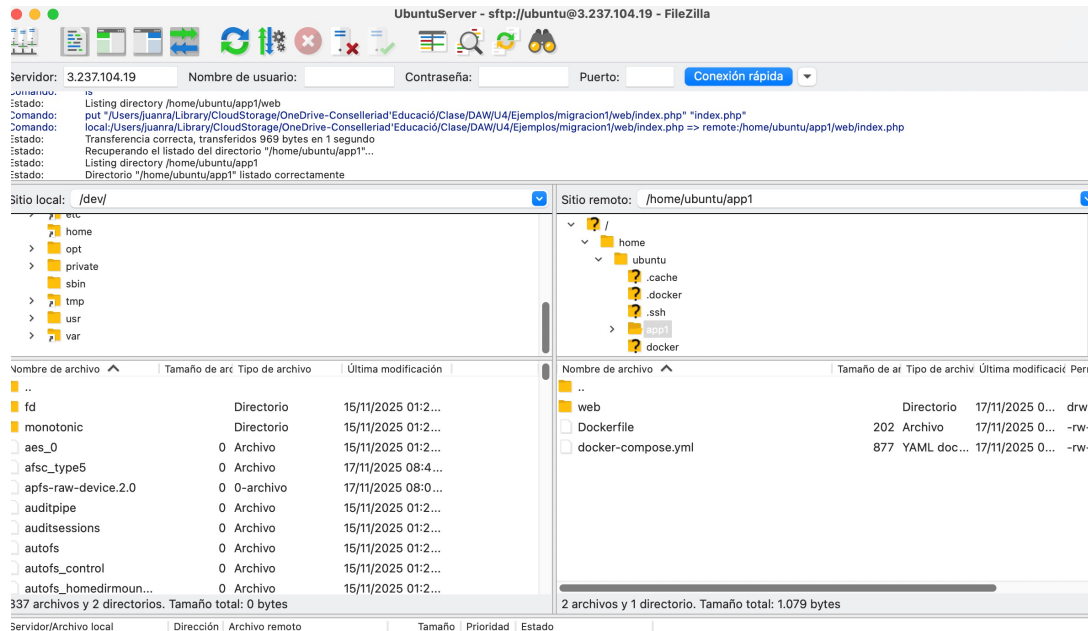
```
db:
  image: mysql:8.0
  container_name: mysql_db
  environment:
    MYSQL_ROOT_PASSWORD: secret
    MYSQL_DATABASE: usuarios
  volumes:
    - db_data:/var/lib/mysql
  restart: always

> Run Service
phpmyadmin:
  image: phpmyadmin/phpmyadmin
  container_name: phpmyadmin
  environment:
    PMA_HOST: db
  ports:
    - "8081:80" # PMA
  depends_on:
    - db
  restart: always
```

Práctica 15 - Migración Docker - AWS

- Parte 1 - Preparación del entorno
 - Necesitaremos un EC2 con Ubuntu con Docker Compose funcionando.
 - Nos aseguraremos que tenemos permisos para conectarnos por los puertos de SSH y los previamente definidos en los ficheros (8080 y 8081)
 - Subiremos los ficheros necesarios a través de FTP a una carpeta del servidor.
 - https://juanracollado.github.io/cloud/u2_practiques/#practica-6-connexio-per-ftp-utilitzant-el-filezilla

Práctica 15 - Migración Docker - AWS



```
ubuntu@ip-10-0-1-31:~$ cd app1/  
ubuntu@ip-10-0-1-31:~/app1$ ls  
Dockerfile  docker-compose.yml  web
```

Práctica 15 - Migración Docker - AWS

- Una vez los ficheros están subidos, tan solo es necesario lanzar el *docker compose up* tal y como si estuviésemos en local.
- El sistema empezará a descargar las imágenes y a crear los contenedores

```
ubuntu@ip-10-0-1-31:~/app1$ docker compose up -d
[+] Running 27/34
  phpmyadmin [#####] 160MB / 221.2MB Pulling
  db [#####] 145.8MB / 234.1MB Pulling
```

Práctica 15 - Migración Docker - AWS

- Al finalizar, tendremos los contenedores corriendo y funcionando en los puertos definidos

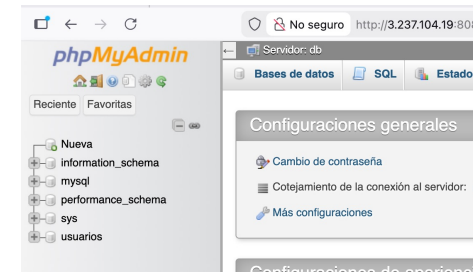
```
[+] Running 6/6
✔ app1-app Built 0.0
✔ Network app1_default Created 0.1
✔ Volume app1_db_data Created 0.0
✔ Container mysql_db Started 0.8
✔ Container phpmyadmin Started 0.9
✔ Container php_app Started 1.0
ubuntu@ip-10-0-1-31:~/app1$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
bedf2a4ee04c   app1-app      "docker-php-entrypoi..." About a minute ago Up About a minute 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   php_app
84aab3c40f95   phpmyadmin/phpmyadmin  "/docker-entrypoint.s..." About a minute ago Up About a minute 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp   phpmyadmin
1a0ef7cb537b   mysql:8.0     "docker-entrypoint.s..." About a minute ago Up About a minute 3306/tcp, 33060/tcp                  mysql_db
```

← → ↻ No seguro http://3.237.104.19:8080

Conexión PDO correcta a la base de datos 'usuarios'

Listado de personas:

- Anna



Práctica 15 - Migración Docker - AWS

- ¿Y qué pasa si tenía datos en la BD que quiero migrar?
 - No suele ser un escenario normal.
 - La BBDD de cada entorno suele tener unos datos
 - Los datos que utilizas para pruebas no van a estar en producción, por ejemplo
 - Si aún así, necesitásemos pasar los datos de la BBDD de un entorno a otro la ÚNICA opción posible es mediante exportar-importar.
 - NUNCA lo haremos copiando los ficheros de /var/lib/mysql ni similar. Esto rompería la BBDD.

¿Dudas?

